

NITTANY DATA LAB

NDL

Convolutional Neural Networks & Transformers

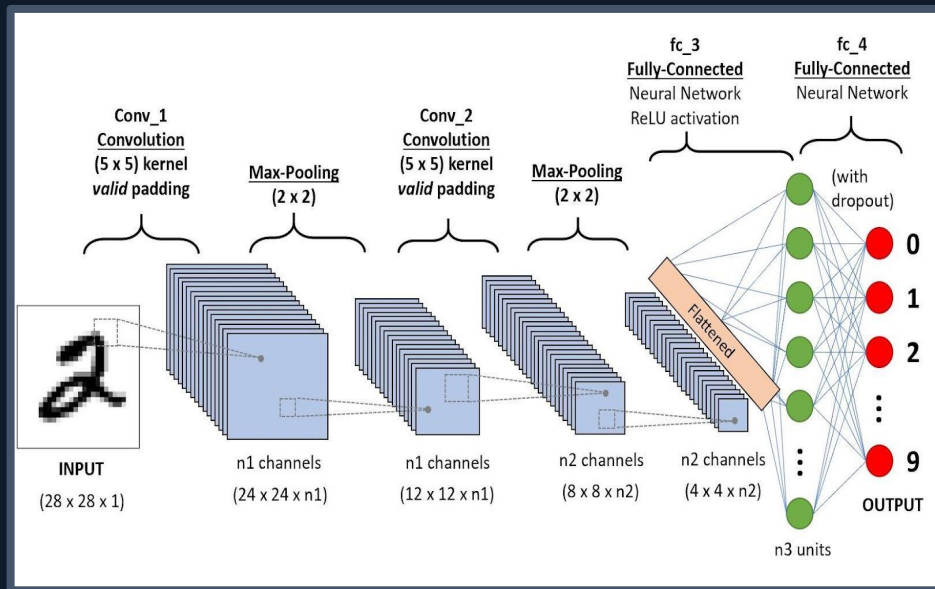


ATTENDANCE - “Transformers”



What are CNNs?

- CNNs process grid-like data, detecting edges, shapes, textures & corners
- CNN's reduce dimensionality of complex data while extracting important features
- Automatically learn relevant features through training
- Excel at image, video, audio & NLP tasks



Early layers: edges & shapes → Deeper layers: objects & faces

CNN Use Cases

01

Image Recognition

Facial recognition, object detection, photo tagging

02

Medical Imaging

Cancer detection, X-ray analysis, MRI scan interpretation

03

Autonomous Vehicles

Road detection, pedestrian tracking, traffic sign recognition

3 Key Layer Types

0 Convolutional Layer

1 Applies filters (kernels) to the input, sliding across the data to create feature maps that capture local patterns. Each filter detects one type of pattern.

0 Pooling Layer

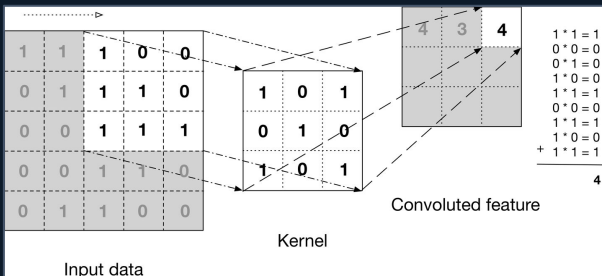
2 Reduces the dimensionality of the data (downsampling). Simplifies information, reduces computational load, and helps the network become position-invariant.

0 Fully Connected Layer

3 Uses the high-level features extracted by previous layers to perform final classification. Each neuron connects to every neuron in the next layer.

Convolutional Layer & Kernels

- Convolution kernels help isolate key features of complex data
- A kernel is a small matrix with randomized initial values that is multiplied with values of subsection of the original data
- Values (weights) are learned during training
- Kernels slide (convolve) across the image to detect patterns
- Output of applying a kernel to an image is called a **feature map**

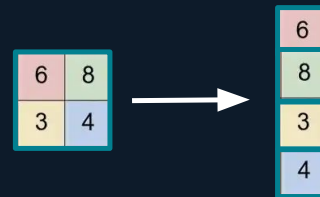
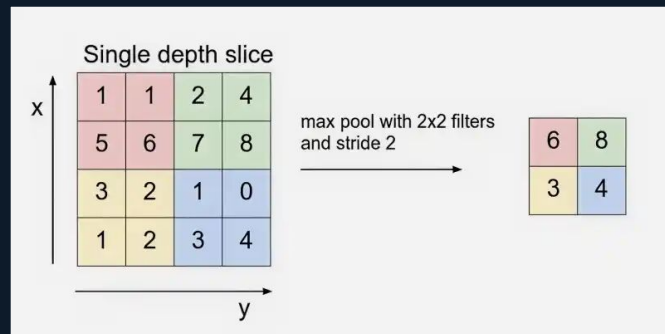


Layer 1	Edges & Gradients
Layer 2	Shapes & Textures
Layer 3	Object Parts (eyes, wheels)
Layer 4+	Full Objects (cats, cars)

Deeper layers combine simple patterns into complex detections

Pooling & Flattening

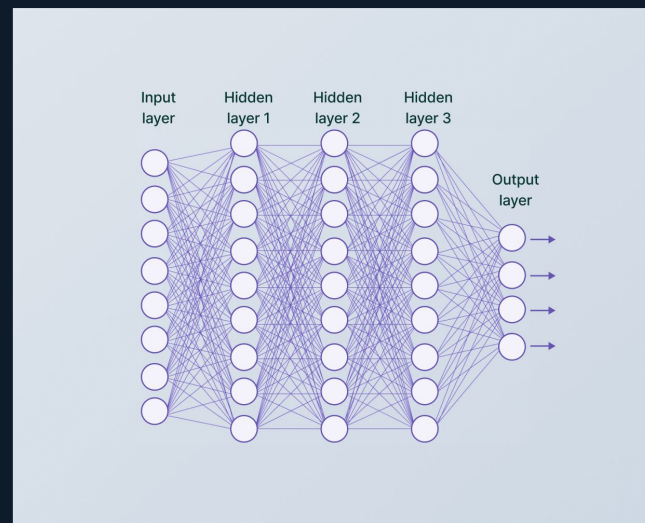
- New feature maps are often still quite large
- Pooling reduces size of these feature maps
- Max-pooling, average-pooling, etc.
- Flattening converts new, smaller feature map into flat, one-dimensional vector
- Allows for classification/traditional modeling



**E.g. 2x2 matrix
flattens to 4x1 vector**

Fully-Connected Layers

- Neural network layer where every neuron in the layer is connected to every neuron in the previous and subsequent layers
- Neural network then assigns weights to flattened feature mapping and classifies
- Uses feedforward neural network, as discussed in our previous lab



Feedforward Neural Network vs Convolutional Neural Network

FNN

Fully connected layers only

Treats each pixel independently

Millions of params for images

Poor at image tasks without preprocessing

Good for tabular / structured data

CNN

Convolutional + Pooling + FC layers

Captures spatial relationships between pixels

Weight sharing reduces parameters dramatically

Automatically extracts relevant features

Optimized for images, video, audio

Lab: CNN for Surface Crack Detection

Using the Kaggle Surface Crack Detection dataset

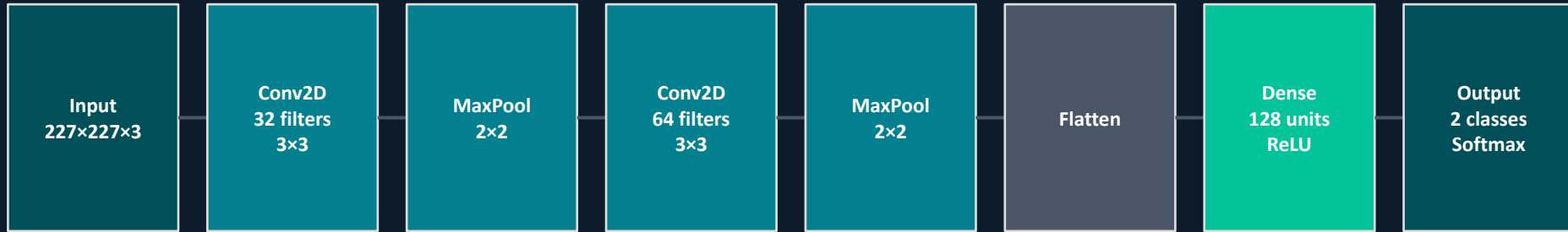
Dataset Info

- 40,000 RGB images (227×227 px)
- 2 classes: Positive (crack) / Negative
- 20,000 images per class (balanced)
- Loaded via kagglehub
- Real-world concrete surface photos

Model Pipeline

- 1 Load & preprocess images
- 2 Build CNN architecture
- 3 Train with backpropagation
- 4 Evaluate on test set
- 5 Visualize feature maps

CNN Architecture for Crack Detection



Conv2D

Learns spatial features via trainable filters; each filter produces one feature map

MaxPool

Selects max value in each 2x2 region, halving spatial dimensions

Dense + Softmax

Flattened features fed into fully connected layers for binary classification

COLAB LINK



Transformer Networks

Parallel Processing

All tokens processed simultaneously — unlike RNNs which are sequential

Contextual Understanding

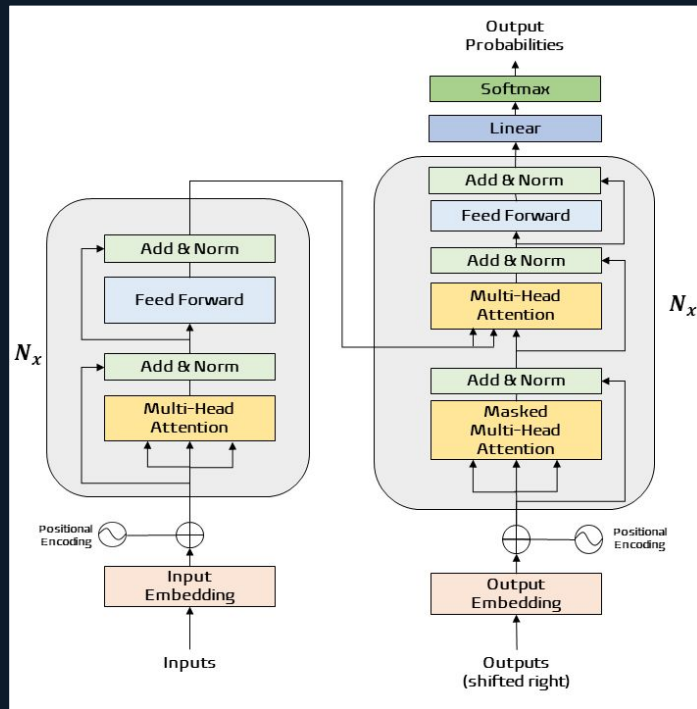
Uses multiple heads of self-attention to understand contextual relationships

Scalability

Performance scales well with more data and larger model size

Versatility

Applied to text, images, audio, code, and time series



Self-Attention Mechanism



Query

What the token is looking for



Key

What information the token contains

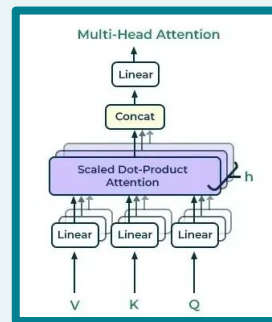


Value

The actual information passed forward

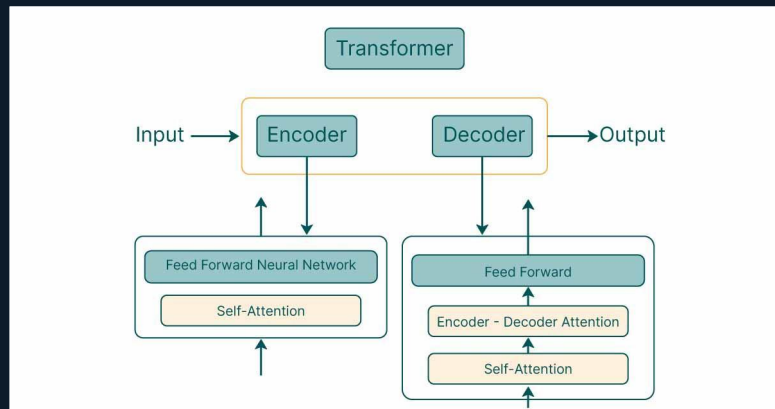
How it works:

1. Each token embedding is transformed into Q, K, V vectors
2. Attention score = similarity between Q and K vectors
3. Scores determine how strongly tokens attend to each other
4. Output = weighted sum of V vectors (hyperparameters of network)



Decoding & Output

- Attention head is fed with input embedding and output embedding
- Decoder output is sent to linear layer (neural network)
- Masks future word tokens to predict the next word
- Builds upon current output with more tokens



Key Takeaways

Convolutional Neural Networks

- Process grid-like data (images, video, audio)
- Kernels learn patterns from edges → complex objects
- Convolutional + Pooling + FC layers
- Weight sharing makes them parameter-efficient
- Power modern computer vision applications

Transformer Networks

- Process all tokens in parallel (not sequentially)
- Self-attention captures long-range dependencies
- Q, K, V vectors compute token relationships
- Multi-head attention learns different relationship types
- Foundation of GPT, Claude, and modern LLMs

Thank You For Attending!

Next Meeting:

4/5 - AgenticAI Workshop